The AI/ML Interview Book

From Scratch to Pro — Theory, Code, and Interview Q&A

ch14_cnns

AYuSh

Contents

Chapter 14: Convolutional Neural Networks

A fully-connected layer treats pixel (0,0) and pixel (100,100) as unrelated inputs and pays for that ignorance in parameters — Listing 1 computes the bill: mapping a 224×224×3 image to 64 feature maps costs a dense layer ~5×10^{11} weights and a 3×3 convolution 1,792. Convolution wins by encoding two facts about images directly into the architecture: *locality* (nearby pixels relate; distant ones mostly don't) and *translation equivariance* (a cat detector should work in any corner). Those built-in assumptions — the convolutional *inductive bias* — are why CNNs learn from thousands of images what dense networks couldn't learn from millions, and the decade of architectures from LeNet to EfficientNet is a sequence of increasingly clever ways to spend the parameter budget convolution saves.

The chapter builds everything: the convolution operation with kernels, stride, padding, and the output-size formula verified against a from-scratch implementation, plus a hand-set kernel acting as an edge detector (Listing 1); receptive-field arithmetic through a stacked network and pooling's mechanics and measured invariance (Listing 2); a complete CNN — im2col convolution, maxpool with argmax-routed gradients, trained end-to-end on digits (Listing 3); the classic architectures told through parameter arithmetic — VGG's small-kernel argument, ResNet's bottleneck, Inception's branches, MobileNet's depthwise separation (Listing 4); residual connections measured (gradient reaching layer 1) and demonstrated (16-layer plain net stuck at 52%, residual twin at 98%) (Listing 5); and transfer learning with its freeze-vs-fine-tune decision staged honestly (Listing 6).

The convolution operation

A convolutional layer slides a small learned kernel across the input, computing at each position the dot product between the kernel and the patch under it — one number per position, forming a **feature map**. (Deep learning's "convolution" is technically cross-correlation — the kernel isn't flipped — a pedantic point worth one sentence in an interview.) The kernel *is* a pattern detector: Listing 1 hand-sets a Sobel kernel and watches it fire exactly on the dark-to-bright boundary column and nowhere else. A layer learns C_{out} such kernels, each spanning all C_{in} input channels, so its weight tensor is $C_{out} \times C_{in} \times K \times K$ and its parameter count $C_{out}(C_{in}K^2 + 1)$ — independent of image size, which is the whole trick: the same 3×3 detector is reused at every spatial position (**weight sharing**), and it only looks at a $K \times K$ neighborhood (**local connectivity**).

The output-size formula — memorize it, it is asked constantly:

$$O = \frac{H - K + 2P}{S} + 1$$

with input size H, kernel K, padding P, stride S. Listing 1 verifies it against the naive implementation for four configurations including AlexNet-style 224/7/2/3 → 112. **Padding** exists to control output size and edge treatment: "same" padding ($P = (K-1)/2$ for odd K, stride 1) preserves spatial size so networks can go deep without shrinking to nothing; "valid" (no padding) shrinks by $K - 1$ per layer. **Stride** downsamples during convolution — stride 2 halves resolution and quarters compute, and modern nets often prefer strided conv to pooling for learned downsampling. Three-dimensional bookkeeping completes the picture: input $H \times W \times C_{in}$, output $O_H \times O_W \times C_{out}$, each output channel a different learned detector bank.

Receptive field — the input region that can influence one output unit — grows with depth: $r_l = r_{l-1} + (K-1)\prod_{i<l} S_i$. Listing 2 tracks it through a VGG-style stack: two 3×3 convs see 5×5; after two 2×2 pools and more convs, a unit sees 24×24 of the input. Two consequences: deep features integrate context (a "face" unit needs to see a face-sized region), and stacking two 3×3 convs gives the same 5×5 field as one 5×5 kernel with fewer parameters (73,856 vs 102,464 at 64 channels) and two nonlinearities instead of one — the argument that made VGG all-3×3 and every network since follow.

Pooling

Pooling summarizes each small window into one value: **max pooling** keeps the strongest activation (did the pattern occur anywhere in this window?), **average pooling** the mean. A 2×2/stride-2 pool halves each spatial dimension, quartering the positions downstream layers process. It has **zero parameters**; its backward pass routes the incoming gradient to the argmax position (max) or spreads it uniformly (average) — Listing 3 implements the argmax routing inside a training loop. Pooling also buys a measure of local translation invariance: Listing 2 shifts random images one pixel and finds 0% of raw pixels unchanged but 34% of maxpool outputs unchanged — the max survives whenever the peak stays inside its window. The modern status: aggressive pooling is out of fashion in classification backbones (strided convs learn their own downsampling; global average pooling at the end replaces giant dense heads — GoogLeNet's move that saved millions of parameters), but max pooling persists where cheap invariance matters.

Classic architectures: the ideas that survived

LeNet-5 (1998) set the template — conv, pool, conv, pool, dense — proving learned convolutional features on digit recognition (~60K parameters). **AlexNet** (2012) scaled the template to ImageNet with GPUs, ReLU (Chapter 12's activation revolution began here), dropout, and augmentation: 60M parameters, top-1 ~63%, and the result that restarted deep learning. **VGG-16** (2014) standardized depth-by-repetition: only 3×3

convs stacked in blocks (the Listing 4 arithmetic), 138M parameters — most of them in three giant dense layers, a design mistake everyone learned from. **GoogLeNet/ Inception** (2014) went the opposite way: **Inception blocks** run 1×1, 3×3, 5×5, and pooling branches *in parallel* and concatenate — the network picks its own kernel mix — with 1×1 convolutions compressing channels before the expensive branches; global average pooling replaced the dense head, landing at 6.8M parameters (Listing 4 prices one block at 164K).

ResNet (2015) solved depth itself. Plain stacks past ~20 layers *degrade* — worse *training* error, so not overfitting but an optimization failure. The **skip connection** $h_{l+1} = h_l + F(h_l)$ reframes each block as learning a *residual* correction to identity rather than a full transformation. Why it works, in three registers the interviewer may probe: (1) *gradient flow* — the Jacobian is $I + \partial F/\partial h$, so backward signal reaches early layers through the identity term regardless of F (Listing 5 measures it: gradient reaching layer 1 of a 40-layer Xavier-init stack is 10^{-6} of the output gradient in the plain net, amplified through the additive paths in the residual one); (2) *ease of optimization* — learning "do nothing plus a correction" is easy where learning an identity through stacked nonlinear layers is hard, so extra layers can't hurt the way they did; (3) *ensemble view* — a residual net is an implicit ensemble over 2^L paths of different lengths, mostly short ones. Listing 5's training experiment is decisive: a 16-hidden-layer plain MLP stalls at 52.2% while its residual twin reaches 98.0% under the same budget. The **bottleneck block** (1×1 down, 3×3, 1×1 up) makes 50+ layer versions affordable — Listing 4: 70K vs 590K parameters, 8× cheaper. **EfficientNet** (2019) closed the era by tuning the *scaling recipe* itself: compound-scale depth, width, and resolution together (each ~fixed ratio per step), atop a mobile-style depthwise-separable backbone — 5.3M parameters matching or beating much larger nets.

1×1 and depthwise separable convolutions

A **1×1 convolution** looks pointless — a kernel with no spatial extent — until you remember channels: at each pixel it computes a linear map $\mathbb{R}^{C_{in}} \to \mathbb{R}^{C_{out}}$, i.e. a per-position fully-connected layer across channels. Uses: channel reduction before expensive ops (Inception's compressions, ResNet bottlenecks — Listing 4 prices 256→64 at 16K parameters), channel expansion, adding nonlinearity depth cheaply (1×1 + ReLU), and replacing dense heads (with global average pooling). **Depthwise separable convolution** (MobileNet, Xception) factorizes a standard conv into *spatial* filtering — one $K \times K$ kernel per input channel, no channel mixing — followed by a 1×1 *pointwise* conv that mixes channels. Listing 4: a 3×3, 128→256 layer drops from 295K parameters to 34K — 8.6× cheaper in parameters and FLOPs — for usually a small accuracy cost. The factorization insight (spatial correlations and cross-channel correlations can be modeled separately) is the same rank-restriction move as SVD compression (Chapter 1) and shows up again in transformer efficiency work.

Transfer learning and fine-tuning

Early conv layers learn edges, textures, color blobs — features so generic they transfer across nearly any visual task; later layers grow increasingly task-specific. Transfer learning exploits this: start from a network pretrained on a large source dataset (ImageNet classically; web-scale contrastive pretraining like CLIP now), and adapt to the target task instead of training from random init. The **strategy ladder**, in increasing data requirements: (1) *frozen feature extractor* — chop the head, run the backbone as a fixed featurizer, train only a new head (linear probe); (2) *partial fine-tuning* — unfreeze the last block(s); (3) *full fine-tuning at reduced LR* (~10× lower, sometimes layerwise-decayed so early generic layers move least); (4) *train from scratch* — only with abundant data. Rules of thumb: the smaller the target dataset, the more you freeze; the more the target domain differs from the source (medical, satellite, spectrograms), the deeper you must fine-tune, because even mid-level source features stop matching.

Listing 6 stages the decision with an honest outcome. Source task: digits 0–4 (~900 samples); target: digits 5–9 with only 15 examples per class, made hard by 448 noise dimensions appended to the input. Fine-tuning the pretrained network at low LR wins (59.4%) over training from scratch (53.6%) — the pretrained layers already know which input dimensions carry signal. But *freezing* the first layer loses to fine-tuning (51.1%): features tuned to 0–4 shapes must adapt to 5–9 shapes, and freezing forbids it. That is the real trade-off, not a slogan: **freeze when source and target domains are close and data is tiny; fine-tune at low LR when the distribution shifts** — and always try the frozen linear probe first, because it's nearly free and calibrates how far you have to go. Practical footnotes: keep (or re-estimate) BN running statistics on the target domain — stale source statistics are a classic silent transfer bug; match the source preprocessing exactly; and when the target set is tiny, augmentation (Chapter 13) compounds with transfer.

Code implementations

Listing 1 — Convolution: kernels, stride, padding, output sizes, parameter sharing

```
"""Listing 1: the convolution operation -- kernels, stride, padding, output sizes."""
import numpy as np
rng = np.random.default_rng(0)

def conv2d(x, k, stride=1, pad=0):
    """Naive 2-D cross-correlation (what DL frameworks call 'convolution')."""
    if pad: x = np.pad(x, pad)
    H, W = x.shape; kh, kw = k.shape
    oh, ow = (H - kh)//stride + 1, (W - kw)//stride + 1
    out = np.zeros((oh, ow))
    for i in range(oh):
        for j in range(ow):
            patch = x[i*stride:i*stride+kh, j*stride:j*stride+kw]
            out[i, j] = (patch * k).sum()               # elementwise multiply + sum
```

```
    return out

# output-size formula: O = (H - K + 2P)/S + 1
print("output-size formula O = (H - K + 2P)/S + 1:")
for H, K, S, P in [(32,3,1,0), (32,3,1,1), (32,5,2,2), (224,7,2,3)]:
    x, k = np.zeros((H,H)), np.zeros((K,K))
    o = conv2d(x, k, S, P).shape[0]
    print(f"  H={H:<4}K={K} S={S} P={P}:  formula {(H-K+2*P)//S+1:<4} actual {o}")

# a hand-set kernel IS a feature detector: vertical edges
img = np.zeros((8, 8)); img[:, 4:] = 1.0              # left dark, right bright
sobel_v = np.array([[-1,0,1],[-2,0,2],[-1,0,1]], float)
resp = conv2d(img, sobel_v, pad=1)
print("\nimage (dark|bright):            vertical-edge response (row 3):")
print(f"  {img[3].astype(int)}   {resp[3].astype(int)}")
print("kernel fires exactly at the boundary column -- convolution = pattern matching")

# parameter sharing: conv vs dense on the same mapping
H, C_in, C_out, K = 224, 3, 64, 3
conv_params = C_out * (C_in * K * K + 1)
dense_params = (H*H*C_in) * (H*H*C_out)
print(f"\n224x224x3 -> 224x224x64: conv 3x3 = {conv_params:,} params,"
      f" dense equivalent = {dense_params:.1e}")
```

Output:

```
output-size formula O = (H - K + 2P)/S + 1:
  H=32  K=3 S=1 P=0:  formula 30   actual 30
  H=32  K=3 S=1 P=1:  formula 32   actual 32
  H=32  K=5 S=2 P=2:  formula 16   actual 16
  H=224 K=7 S=2 P=3:  formula 112  actual 112

image (dark|bright):            vertical-edge response (row 3):
  [0 0 0 0 1 1 1 1]   [ 0  0  0  4  4  0  0 -4]
kernel fires exactly at the boundary column -- convolution = pattern matching

224x224x3 -> 224x224x64: conv 3x3 = 1,792 params, dense equivalent = 4.8e+11
```

The formula matches the sliding-window implementation in all four configurations; a hand-set Sobel kernel fires only at the edge; and weight sharing turns a 10^{11}-parameter dense mapping into 1,792.

Listing 2 — Receptive field arithmetic and pooling

```
"""Listing 2: receptive field growth and pooling."""
import numpy as np

# receptive field recurrence: r_out = r_in + (k-1)*j_in ; j_out = j_in*s
print("stack of layers -> receptive field (r) and jump (j):")
layers = [("conv3x3 s1", 3, 1), ("conv3x3 s1", 3, 1), ("maxpool2x2 s2", 2, 2),
          ("conv3x3 s1", 3, 1), ("conv3x3 s1", 3, 1), ("maxpool2x2 s2", 2, 2),
          ("conv3x3 s1", 3, 1)]
r, j = 1, 1
for name, k, s in layers:
    r = r + (k - 1) * j
    j = j * s
    print(f"  after {name:<15} r = {r:>3} px   j = {j}")
print("two stacked 3x3 convs see 5x5 (like one 5x5 kernel) with 18k+2 weights vs 25k+1"
      "\nand two nonlinearities -- the VGG argument for small kernels")

# pooling: downsampling + local translation invariance
```

```
x = np.array([[1,3,2,0],[4,8,1,1],[2,1,7,2],[0,3,2,9]], float)
def pool(x, f):     # 2x2, stride 2
    return np.array([[f(x[i:i+2, j:j+2]) for j in (0,2)] for i in (0,2)])
print(f"\ninput:\n{x.astype(int)}")
print(f"maxpool 2x2:\n{pool(x, np.max).astype(int)}")
print(f"avgpool 2x2:\n{pool(x, np.mean)}")

rng = np.random.default_rng(0)                          # measure invariance
statistically
imgs = rng.random((2000, 4, 5))                         # extra column so shift needs no
padding
raw_same, pool_same = [], []
for im in imgs:
    a, b = im[:4, :4], im[:4, 1:5]                      # original vs 1px-shifted crop
    raw_same.append((a == b).mean())
    pool_same.append((pool(a, np.max) == pool(b, np.max)).mean())
print(f"\n1px shift, 2000 random images: raw pixels unchanged {np.mean(raw_same)*100:.
0f}%,"
      f" maxpool outputs unchanged {np.mean(pool_same)*100:.0f}%")
print("pooling has ZERO parameters; backward routes gradient to the argmax (max) or
spreads it (avg)")
```

Output:

```
stack of layers -> receptive field (r) and jump (j):
  after conv3x3 s1      r =   3 px   j = 1
  after conv3x3 s1      r =   5 px   j = 1
  after maxpool2x2 s2   r =   6 px   j = 2
  after conv3x3 s1      r =  10 px   j = 2
  after conv3x3 s1      r =  14 px   j = 2
  after maxpool2x2 s2   r =  16 px   j = 4
  after conv3x3 s1      r =  24 px   j = 4
two stacked 3x3 convs see 5x5 (like one 5x5 kernel) with 18k+2 weights vs 25k+1
and two nonlinearities -- the VGG argument for small kernels

input:
[[1 3 2 0]
 [4 8 1 1]
 [2 1 7 2]
 [0 3 2 9]]
maxpool 2x2:
[[8 2]
 [3 9]]
avgpool 2x2:
[[4.  1. ]
 [1.5 5. ]]

1px shift, 2000 random images: raw pixels unchanged 0%, maxpool outputs unchanged 34%
pooling has ZERO parameters; backward routes gradient to the argmax (max) or spreads it
(avg)
```

The recurrence $r \leftarrow r + (K-1)\cdot j$, $j \leftarrow j \cdot S$ tracks how each layer widens what a unit sees — downsampling (jump) is what makes receptive fields grow fast. The invariance measurement is honest: maxpool preserves a third of outputs under a shift that changes every raw pixel.

Listing 3 — A complete CNN from scratch (im2col), trained

```
"""Listing 3: a complete CNN from scratch (im2col) trained on digits."""
import numpy as np
from sklearn.datasets import load_digits
```

```
from sklearn.model_selection import train_test_split
rng = np.random.default_rng(1)

digits = load_digits()
X = (digits.data/16.0).reshape(-1, 1, 8, 8)            # (n, C=1, 8, 8)
Xtr, Xte, ytr, yte = train_test_split(X, digits.target, test_size=0.25,
                                      random_state=0, stratify=digits.target)

def im2col(x, k):                                       # x: (n, C, H, W) -> patches
matrix
    n, C, H, W = x.shape; o = H - k + 1
    cols = np.empty((n, C*k*k, o*o))
    idx = 0
    for i in range(o):
        for j in range(o):
            cols[:, :, idx] = x[:, :, i:i+k, j:j+k].reshape(n, -1)
            idx += 1
    return cols                                         # (n, C*k*k, o*o)

def relu(z): return np.maximum(0, z)
def softmax(z):
    e = np.exp(z - z.max(1, keepdims=True)); return e/e.sum(1, keepdims=True)

F, K = 8, 3                                             # 8 filters of 3x3
Wc = rng.normal(0, np.sqrt(2/(K*K)), (F, K*K))          # conv weights as a matrix
bc = np.zeros(F)
o = 8 - K + 1                                           # 6x6 feature maps -> pool to
3x3
Wd = rng.normal(0, np.sqrt(2/(F*3*3)), (F*3*3, 10)); bd = np.zeros(10)

def forward(xb):
    cols = im2col(xb, K)                                # (n, 9, 36)
    conv = relu(np.einsum("fk,nkp->nfp", Wc, cols) + bc[:, None])   # (n, F, 36)
    fmap = conv.reshape(-1, F, o, o)
    pooled = fmap.reshape(-1, F, 3, 2, 3, 2).max(axis=(3, 5))       # 2x2 maxpool
    flat = pooled.reshape(len(xb), -1)
    return cols, conv, fmap, pooled, flat, softmax(flat @ Wd + bd)

lr = 0.2
for ep in range(15):
    order = rng.permutation(len(Xtr))
    for s in range(0, len(Xtr), 64):
        idx = order[s:s+64]; xb, yb = Xtr[idx], ytr[idx]
        cols, conv, fmap, pooled, flat, P = forward(xb)
        n = len(idx)
        dz = P.copy(); dz[np.arange(n), yb] -= 1; dz /= n
        dWd, dbd = flat.T @ dz, dz.sum(0)
        dflat = (dz @ Wd.T).reshape(-1, F, 3, 3)
        # maxpool backward: route gradient to argmax within each 2x2 window
        dfmap = np.zeros_like(fmap)
        win = fmap.reshape(-1, F, 3, 2, 3, 2)
        mx = pooled[:, :, :, None, :, None]
        dfmap = ((win == mx) * dflat[:, :, :, None, :, None]).reshape(fmap.shape)
        dconv = dfmap.reshape(-1, F, o*o) * (conv > 0)
        dWc = np.einsum("nfp,nkp->fk", dconv, cols)
        dbc = dconv.sum(axis=(0, 2))
        Wd -= lr*dWd; bd -= lr*dbd; Wc -= lr*dWc; bc -= lr*dbc
    if ep % 7 == 0 or ep == 14:
        acc = (forward(Xte)[-1].argmax(1) == yte).mean()
        print(f"epoch {ep+1:2d}: test acc {acc:.4f}")
print(f"\nconv layer: {Wc.size + F} params; learned filters have structure --")
print("filter 0, 3x3:", np.round(Wc[0].reshape(3,3), 2).tolist())
```

Output:

```
epoch  1: test acc 0.2356
epoch  8: test acc 0.9333
epoch 15: test acc 0.9511

conv layer: 80 params; learned filters have structure --
filter 0, 3x3: [[0.75, 1.36, 0.32], [-1.38, 1.31, 0.5], [-0.63, 0.87, 0.48]]
```

im2col unrolls every patch into a column so convolution becomes one matrix multiply — exactly how frameworks implement it. The maxpool backward routes gradients through a boolean argmax mask. Eight 3×3 filters — 80 parameters — carry the digits task to 95.1%, and the learned filter shows oriented structure (a left-edge/right-edge contrast), not noise.

Listing 4 — Architecture ideas priced in parameters

```python
"""Listing 4: architecture ideas by parameter count -- VGG stacks, 1x1, depthwise,
Inception."""
import numpy as np

def conv_params(cin, cout, k): return cout * (cin * k * k + 1)

print("== small kernels (VGG): one 5x5 vs two 3x3, 64->64 channels")
print(f"  one 5x5 : {conv_params(64,64,5):,} params, 1 nonlinearity, RF 5")
print(f"  two 3x3 : {2*conv_params(64,64,3):,} params, 2 nonlinearities, RF 5")

print("\n== 1x1 convolution = per-pixel linear layer across channels")
print(f"  256->64 channel reduction with 1x1: {conv_params(256,64,1):,} params")
print(f"  bottleneck 256 -1x1-> 64 -3x3-> 64 -1x1-> 256:"
      f" {conv_params(256,64,1)+conv_params(64,64,3)+conv_params(64,256,1):,}")
print(f"  naive 3x3 at 256 channels          : {conv_params(256,256,3):,}"
      "   <- bottleneck saves 8x")

print("\n== depthwise separable (MobileNet): spatial and channel mixing split")
cin, cout, k, hw = 128, 256, 3, 56
std   = conv_params(cin, cout, k)
dw    = cin * (k*k + 1)                      # one kxk filter PER channel
pw    = conv_params(cin, cout, 1)            # then 1x1 to mix channels
print(f"  standard 3x3 {cin}->{cout}: {std:,} params, {std*hw*hw/1e9:.2f} GFLOP-ish")
print(f"  depthwise+pointwise    : {dw+pw:,} params, {(dw+pw)*hw*hw/1e9:.2f}"
      f"  ({std/(dw+pw):.1f}x cheaper)")

print("\n== Inception: parallel branches, concatenated")
b = conv_params(192,64,1) + conv_params(192,96,1)+conv_params(96,128,3) \
  + conv_params(192,16,1)+conv_params(16,32,5) + conv_params(192,32,1)
print(f"  inception-3a block: {b:,} params across 1x1 / 3x3 / 5x5 / pool-proj
branches")
print("  network chooses its own kernel size mix; 1x1s keep every branch cheap")

print("\n== parameter totals (approx, ImageNet classifiers)")
for name, p, top1 in [("LeNet-5 (1998)", "60K", "MNIST era"), ("AlexNet (2012)",
"60M", "63%"),
                      ("VGG-16 (2014)", "138M", "72%"), ("GoogLeNet (2014)", "6.8M",
"70%"),
                      ("ResNet-50 (2015)", "25.6M", "76%"), ("EfficientNet-B0 (2019)",
"5.3M", "77%")]:
    print(f"  {name:<23}{p:>7}   top-1 {top1}")
```

Output:

```
== small kernels (VGG): one 5x5 vs two 3x3, 64->64 channels
  one 5x5 : 102,464 params, 1 nonlinearity, RF 5
  two 3x3 : 73,856 params, 2 nonlinearities, RF 5

== 1x1 convolution = per-pixel linear layer across channels
  256->64 channel reduction with 1x1: 16,448 params
  bottleneck 256 -1x1-> 64 -3x3-> 64 -1x1-> 256: 70,016
  naive 3x3 at 256 channels          : 590,080   <- bottleneck saves 8x

== depthwise separable (MobileNet): spatial and channel mixing split
  standard 3x3 128->256: 295,168 params, 0.93 GFLOP-ish
  depthwise+pointwise    : 34,304 params, 0.11  (8.6x cheaper)

== Inception: parallel branches, concatenated
  inception-3a block: 163,696 params across 1x1 / 3x3 / 5x5 / pool-proj branches
  network chooses its own kernel size mix; 1x1s keep every branch cheap

== parameter totals (approx, ImageNet classifiers)
  LeNet-5 (1998)             60K   top-1 MNIST era
  AlexNet (2012)             60M   top-1 63%
  VGG-16 (2014)             138M   top-1 72%
  GoogLeNet (2014)          6.8M   top-1 70%
  ResNet-50 (2015)         25.6M   top-1 76%
  EfficientNet-B0 (2019)     5.3M   top-1 77%
```

Every architectural fashion is parameter arithmetic: VGG's stacked 3×3s buy the same receptive field cheaper; 1×1 bottlenecks cut a block 8×; depthwise separation cuts a layer 8.6×; and the history table shows the field learning to do more with less — EfficientNet matches 26× more parameters than AlexNet used.

Listing 5 — Residual connections: measured gradient flow, then trainability

```python
"""Listing 5: residual connections -- gradient flow and trainability, measured."""
import numpy as np
rng = np.random.default_rng(2)

# (a) backward signal through 40 layers: plain vs residual
def grad_ratio(residual, depth=40, width=64):
    # Xavier (not He) on purpose: a mediocre init, so depth exposes the difference
    Ws = [rng.normal(0, np.sqrt(1/width), (width, width)) for _ in range(depth)]
    h = rng.normal(size=(64, width)); zs = []
    for W in Ws:
        z = h @ W; a = np.maximum(0, z)
        h = h + a if residual else a; zs.append(z)
    g = np.ones((64, width)) / (64*width)
    g_out = np.linalg.norm(g)
    for l in range(depth-1, -1, -1):
        gb = (g * (zs[l] > 0)) @ Ws[l].T        # through the branch
        g = g + gb if residual else gb          # residual: identity path ADDS
    return np.linalg.norm(g) / g_out            # signal reaching the INPUT

print("||grad at input|| / ||grad at output||, 40 layers, Xavier init:")
print(f"  plain    : {grad_ratio(False):.2e}")
print(f"  residual : {grad_ratio(True):.2e}   <- identity path keeps early layers
alive")

# (b) does it train? deep plain vs residual MLP on digits
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
digits = load_digits(); X = digits.data/16.0
```

```
Xtr, Xte, ytr, yte = train_test_split(X, digits.target, test_size=0.25,
                                      random_state=0, stratify=digits.target)
def softmax(z):
    e = np.exp(z - z.max(1, keepdims=True)); return e/e.sum(1, keepdims=True)

def train(residual, depth=16, width=64, lr=0.05, epochs=20):
    r = np.random.default_rng(0)
    Win = r.normal(0, np.sqrt(2/64), (64, width))
    Ws = [r.normal(0, np.sqrt(2/width)*(0.1 if residual else 1.0), (width, width))
          for _ in range(depth)]
    Wout = r.normal(0, np.sqrt(2/width), (width, 10))
    for ep in range(epochs):
        for s in range(0, len(Xtr), 64):
            idx = r.permutation(len(Xtr))[s:s+64]
            h = Xtr[idx] @ Win; hs, zs = [h], []
            for W in Ws:
                z = h @ W; a = np.maximum(0, z)
                h = h + a if residual else a
                hs.append(h); zs.append(z)
            P = softmax(h @ Wout)
            dz = P.copy(); dz[np.arange(len(idx)), ytr[idx]] -= 1; dz /= len(idx)
            dWout = h.T @ dz; g = dz @ Wout.T
            for l in range(len(Ws)-1, -1, -1):
                ga = g * (zs[l] > 0)
                dW = hs[l].T @ ga
                gb = ga @ Ws[l].T
                g = g + gb if residual else gb
                Ws[l] -= lr*dW
            Win -= lr*(Xtr[idx].T @ g); Wout -= lr*dWout
            if not np.isfinite(Win).all(): return None
    h2 = Xte @ Win
    for W in Ws:
        a = np.maximum(0, h2 @ W); h2 = h2 + a if residual else a
    return (softmax(h2 @ Wout).argmax(1) == yte).mean()

print(f"\n16 hidden layers on digits, same budget:")
print(f"  plain    : test acc {train(False):.3f}")
print(f"  residual : test acc {train(True):.3f}")
```

Output:

```
||grad at input|| / ||grad at output||, 40 layers, Xavier init:
  plain    : 3.84e-06
  residual : 1.06e+07   <- identity path keeps early layers alive

16 hidden layers on digits, same budget:
  plain    : test acc 0.522
  residual : test acc 0.980
```

Through 40 plain layers the input-reaching gradient is a millionth of the output gradient; the identity paths of the residual version deliver it amplified (the additive growth is why real ResNets pair skips with BN and near-zero branch init — this listing scales branch weights by 0.1 for the same reason). The training run is the punchline: same depth, same budget — 52% vs 98%.

Listing 6 — Transfer learning: freeze vs fine-tune, honestly

```
"""Listing 6: transfer learning -- pretrain on digits 0-4, transfer features to 5-9."""
import numpy as np
from sklearn.datasets import load_digits
```

```
from sklearn.model_selection import train_test_split
rng = np.random.default_rng(3)

digits = load_digits()
X = np.hstack([digits.data/16.0, rng.normal(0, 0.35, (len(digits.data), 448))])
y = digits.target                                    # 64 signal dims + 448 noise
dims
src = y < 5
Xs, ys = X[src], y[src]                              # source: 0-4, ~900 samples
Xt_all, yt_all = X[~src], y[~src] - 5                # target: 5-9, relabeled
Xt, Xt_te, yt, yt_te = train_test_split(Xt_all, yt_all, test_size=0.5,
                                        random_state=0, stratify=yt_all)
keep = np.concatenate([np.where(yt == c)[0][:15] for c in range(5)])
Xt, yt = Xt[keep], yt[keep]                          # 75 target samples

def relu(z): return np.maximum(0,z)
def softmax(z):
    e=np.exp(z-z.max(1,keepdims=True)); return e/e.sum(1,keepdims=True)

D, H1, H2 = X.shape[1], 128, 64
def train(Xa, ya, init=None, freeze1=False, lr=0.1, epochs=150):
    r = np.random.default_rng(0)
    if init is None:
        W1=r.normal(0,np.sqrt(2/D),(D,H1)); b1=np.zeros(H1)
        W2=r.normal(0,np.sqrt(2/H1),(H1,H2)); b2=np.zeros(H2)
    else:
        (W1,b1,W2,b2) = [a.copy() for a in init]
    W3=r.normal(0,np.sqrt(2/H2),(H2,5)); b3=np.zeros(5)
    for ep in range(epochs):
        for s in range(0,len(Xa),32):
            idx=r.permutation(len(Xa))[s:s+32]
            H1a=relu(Xa[idx]@W1+b1); H2a=relu(H1a@W2+b2); P=softmax(H2a@W3+b3)
            dz=P.copy(); dz[np.arange(len(idx)),ya[idx]]-=1; dz/=len(idx)
            W3-=lr*H2a.T@dz; b3-=lr*dz.sum(0)
            dh2=(dz@W3.T)*(H2a>0)
            W2-=lr*H1a.T@dh2; b2-=lr*dh2.sum(0)
            if not freeze1:                           # layer 1 = generic features
                dh1=(dh2@W2.T)*(H1a>0)
                W1-=lr*Xa[idx].T@dh1; b1-=lr*dh1.sum(0)
    return W1,b1,W2,b2,W3,b3

def acc(Xe,ye,W1,b1,W2,b2,W3,b3):
    return (softmax(relu(relu(Xe@W1+b1)@W2+b2)@W3+b3).argmax(1)==ye).mean()

pre = train(Xs, ys, epochs=200)                      # pretrain on plentiful source
print(f"source-task sanity: acc on 0-4 = {acc(Xs, ys, *pre):.3f}")
runs = {
  "from scratch (75 samples)":       train(Xt, yt),
  "freeze layer1, tune rest":        train(Xt, yt, init=pre[:4], freeze1=True,
lr=0.05, epochs=300),
  "fine-tune all, lower LR":         train(Xt, yt, init=pre[:4], lr=0.02),
}
print("target = digits 5-9, 15 examples/class, 448 noise dims in the input:")
for name, m in runs.items():
    print(f"  {name:<28} test acc {acc(Xt_te, yt_te, *m):.4f}")
print("\nfine-tuning wins: pretrained layers know which input dims carry signal,")
print("but 0-4 features must ADAPT to 5-9 shapes -- freezing is too rigid when")
print("source and target differ. Freeze when domains are close and data is tiny;")
print("fine-tune at low LR when the target distribution shifts.")
```

Output:

```
source-task sanity: acc on 0-4 = 1.000
target = digits 5-9, 15 examples/class, 448 noise dims in the input:
  from scratch (75 samples)    test acc 0.5357
  freeze layer1, tune rest     test acc 0.5112
```

```
  fine-tune all, lower LR      test acc 0.5938

fine-tuning wins: pretrained layers know which input dims carry signal,
but 0-4 features must ADAPT to 5-9 shapes -- freezing is too rigid when
source and target differ. Freeze when domains are close and data is tiny;
fine-tune at low LR when the target distribution shifts.
```

Transfer beats scratch by 6 points — the pretrained layers already know which of 512 input dimensions carry signal. But the frozen variant *loses* to fine-tuning because source (0–4) and target (5–9) shapes differ: the freeze-vs-fine-tune decision tracks domain distance, not dogma.

Pitfalls, comparisons and practical tips

Architecture idea → mechanism → cost table:

Idea	Mechanism	What it buys	Cost
3×3 stacking (VGG)	two 3×3 = RF 5	fewer params + extra nonlinearity	more layers to schedule
1×1 conv (NiN/ Inception)	per-pixel channel mixing	cheap compression/ expansion	none — use freely
Bottleneck (ResNet)	1×1 down → 3×3 → 1×1 up	8× cheaper blocks → depth	slight capacity loss per block
Skip connection	identity + residual branch	trainable 100+ layers	memory for activations
Depthwise separable	spatial ⊗ channel factorization	~8–9× cheaper layers	small accuracy dip
Global average pooling	mean over positions → head	kills giant dense layers	none in practice
Compound scaling	depth·width·resolution together	best accuracy/FLOP	search cost (done once)

The recurring pitfalls:

- **Botching the output-size formula under stride.** (H − K + 2P)/S + 1 must divide cleanly or the framework floors it — off-by-one feature maps and shape errors trace here. Compute the whole stack's shapes before training.
- **"Same" padding assumed everywhere.** Valid-padding layers shrink maps; five of them eat 10 pixels of border. Know which convention each layer uses.
- **Forgetting channels in parameter counts.** A "3×3 conv" on 256 channels to 256 channels is 590K parameters, not 9. Interviewers test exactly this.
- **Pooling backward hand-waving.** Max pooling's gradient goes only to the argmax; average spreads uniformly. Saying "it has no gradient because no parameters" confuses parameter gradients with routed input gradients.

- **Reading ResNet degradation as overfitting.** Plain-deep nets have worse *training* error — it is an optimization pathology; skips fix trainability, not variance.
- **Fine-tuning with frozen BN statistics from the source domain.** Re-estimate or unfreeze BN on target data; stale statistics silently distort every activation (the transfer-learning cousin of Chapter 13's eval-mode bug).
- **Transfer with mismatched preprocessing.** The backbone expects the source's normalization and input size; feeding raw pixels wastes the pretraining.
- **Freezing dogmatically.** Listing 6: freezing lost to low-LR fine-tuning the moment the target domain shifted. Run the frozen linear probe as a cheap baseline, then decide.
- **Counting FLOPs by parameters.** Conv FLOPs scale with output positions × kernel volume; a 3×3 at 224^2 costs vastly more compute than a dense layer with equal parameters. Efficiency work optimizes FLOPs and memory traffic, not just weights.

Interview questions and answers

Q1. Why do CNNs need so many fewer parameters than dense networks on images?

Two structural priors: local connectivity (each unit sees a K×K patch, not the whole image) and weight sharing (the same kernel is reused at every position, since a useful detector is useful everywhere). Listing 1: 224×224×3 → 64 maps costs 1,792 conv parameters vs ~5×10^{11} dense. The parameter count C_out(C_in·K^2+1) is independent of image size. This inductive bias also gives translation equivariance: shift the input, the feature map shifts identically.

Q2. Give the output-size formula and compute: 224×224 input, 7×7 kernel, stride 2, padding 3.

O = (H − K + 2P)/S + 1 = (224 − 7 + 6)/2 + 1 = 223/2 + 1 → floor(111.5) + 1 = 112 (frameworks floor the division). Verified in Listing 1. Same padding for stride 1 needs P = (K−1)/2 (odd K). Interviewers use this to check you can lay out a whole network's shapes; practice the composition through several layers.

Q3. Convolution vs cross-correlation — does the difference matter?

True convolution flips the kernel before sliding; deep learning frameworks implement cross-correlation (no flip) and call it convolution. It doesn't matter for learning: the kernel is learned, so it absorbs the flip. It matters for (a) pedantic correctness, (b) implementing/verifying against signal-processing code, (c) transposed convolutions and some initialization schemes where orientation conventions bite.

Q4. What is a receptive field, how do you compute its growth, and why does it matter?

The input region that can influence a given unit. Recurrence: $r \leftarrow r + (K-1)\cdot j$ and $j \leftarrow j\cdot S$, where j (jump) is the cumulative stride. Listing 2 tracks a VGG-ish stack to r=24 px after 7 layers — downsampling is what makes r grow fast. It matters because a feature can only recognize patterns no larger than its receptive field: detecting whole objects needs deep layers or aggressive striding; segmentation nets add dilated convolutions to grow r without losing resolution.

Q5. Why did VGG use only 3×3 kernels?

Two stacked 3×3 convs have a 5×5 receptive field but fewer parameters ($2\cdot 9C^2$ vs $25C^2$ per channel pair — Listing 4: 73,856 vs 102,464 at C=64) and interpose two nonlinearities instead of one, increasing discriminative depth. Three 3×3s similarly replace 7×7. Small kernels + depth became the universal recipe; the exceptions are stem layers (fast early downsampling) and depthwise large-kernel revivals (e.g. 7×7 depthwise in ConvNeXt) where the depthwise trick makes big kernels cheap.

Q6. Max pooling vs average pooling vs strided convolution for downsampling.

Max: keeps strongest response — "did the feature appear in this window"; sharp, sparse gradients (argmax-routed); best for detection-ish invariance. Average: smooth summary, uniform gradient; global average pooling (whole map → one value) is the standard classifier head, replacing giant dense layers. Strided conv: *learned* downsampling with parameters — modern backbones prefer it; costs parameters and can alias. Pooling itself has zero parameters (Listing 2/3 show mechanics and backward routing).

Q7. Explain im2col. Why do frameworks implement convolution this way?

Unroll every K×K×C_in patch of the input into a column of a matrix (Listing 3: (n, $C \cdot K^2$, positions)); convolution then becomes one big matrix multiply between the (C_out, $C \cdot K^2$) weight matrix and the columns. GEMM is the most optimized primitive on GPUs/CPUs (tensor cores, cache-blocked BLAS), so trading memory (patch duplication) for a single large matmul wins. Alternatives — FFT convolution (large kernels), Winograd (small kernels) — exist for special regimes.

Q8. Describe the maxpool backward pass precisely.

Forward caches which position in each window achieved the max (argmax mask). Backward: each pooled output's gradient is routed entirely to that argmax position; all other positions get zero (they didn't influence the output). Ties are broken by convention. Listing 3 implements it as (window == max) * upstream. Average pooling instead distributes gradient/window-size uniformly. No parameters are updated — pooling has none — but gradient must still flow through to earlier conv layers.

Q9. Walk the LeNet → AlexNet → VGG → Inception → ResNet → EfficientNet arc: one idea each.

LeNet: the conv-pool-dense template works (learned features beat hand-crafted). AlexNet: scale it — GPUs, ReLU, dropout, augmentation; deep learning's 2012 restart. VGG: uniform 3×3 depth; simplicity as principle (but 138M params, mostly a dense-head mistake). Inception: parallel multi-scale branches + 1×1 compressions + global average pooling — 6.8M params. ResNet: skip connections make depth itself trainable (52%→98% in Listing 5's miniature). EfficientNet: stop hand-designing — compound-scale depth/width/resolution over an efficient depthwise backbone; 5.3M params, best accuracy/FLOP of its era.

Q10. Why do plain networks degrade past ~20 layers, and why is it not overfitting?

Because *training* error worsens too — an optimization failure, not variance. A deeper net strictly contains the shallower one's solutions (set extra layers to identity), yet SGD cannot find them: identity is hard to represent through stacks of nonlinear layers, and gradient signal degrades multiplicatively. Skip connections make identity the default (F=0) and corrections residual, restoring trainability. Listing 5: 16-layer plain 52.2%, residual twin 98.0%, same everything else.

Q11. Give the three standard explanations for why residual connections work.

(1) Gradient highway: Jacobian I + ∂F/∂h means backward signal reaches any earlier layer additively, bypassing the product of layer Jacobians (Listing 5: 3.8e-6 of the gradient reaches layer 1 of 40 plain layers). (2) Optimization reframing: learning a residual correction to identity is easier than learning full transformations; extra layers can default to no-op. (3) Implicit ensemble: unrolling gives 2^L paths of varying depth, dominated by short paths — deleting single blocks barely hurts, unlike plain nets. All three are correct lenses; citing two crisply is a strong answer.

Q12. What is a bottleneck block and why does ResNet-50 use it?

1×1 conv compresses channels (256→64), 3×3 operates at the narrow width, 1×1 expands back (64→256), wrapped in a skip. Cost: 70K parameters vs 590K for a plain 3×3 at 256 channels (Listing 4) — 8× cheaper, making 50/101/152-layer networks affordable. The 1×1s are per-pixel linear channel maps; the design principle is doing expensive spatial work at low channel width.

Q13. What does a 1×1 convolution do, concretely, and name three uses.

At every spatial position independently, it applies the same linear map across channels: R^{C_in} → R^{C_out} — a fully-connected layer per pixel. Uses: channel reduction/expansion around expensive ops (bottlenecks, Inception compressions — 256→64 costs 16K params, Listing 4); adding nonlinearity depth cheaply (1×1+ReLU); cross-channel feature remixing (network-in-network); building classifier heads with global average pooling instead of dense layers.

Q14. Derive the parameter and FLOP savings of depthwise separable convolution.

Standard: $C_{out} \cdot C_{in} \cdot K^2$ weights. Depthwise separable: $C_{in} \cdot K^2$ (one K×K filter per input channel, no mixing) + $C_{in} \cdot C_{out}$ (pointwise 1×1 mixing). Ratio $\approx 1/C_{out} + 1/K^2$ — for K=3, C_{out} large: ~1/9 + small ≈ 8-9× cheaper; Listing 4: 295K → 34K (8.6×). FLOPs scale identically (same counts × positions). Assumption made explicit: spatial and cross-channel correlations are separable — usually nearly true, costing little accuracy (MobileNet, Xception; EfficientNet's backbone).

Q15. What is compound scaling in EfficientNet?

Prior practice scaled one axis: deeper (ResNet-152), wider (WideResNet), or higher-resolution inputs. EfficientNet observes the three interact — higher resolution needs more depth (receptive field) and width (finer patterns) — and scales all three by fixed ratios (α, β, γ with $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$) raised to a compound coefficient, found by small grid search on the base model. Result: each doubling of FLOPs spends optimally across axes; B0→B7 traces the accuracy/compute frontier of its era.

Q16. When do you freeze pretrained layers vs fine-tune them? What governs the decision?

Two axes: target data size and domain distance from the source. Tiny data + close domain → freeze backbone, train head (linear probe): least overfitting risk. More data or shifted domain → unfreeze progressively (last blocks first) at ~10× lower LR, optionally layerwise-decayed. Far domain (medical, satellite, audio spectrograms) → deep or full fine-tuning; even mid-level source features mismatch. Listing 6 stages a shifted-domain case: fine-tune 59.4% > scratch 53.6% > frozen 51.1% — freezing lost exactly because the domains differed. Always run the frozen probe first as a cheap baseline.

Q17. What silently breaks when fine-tuning a pretrained CNN? Name three gotchas.

(1) BatchNorm statistics: source-domain running means/vars distort target activations — re-estimate them or keep BN trainable (or freeze BN *parameters* but update stats — know which your framework does). (2) Preprocessing mismatch: the backbone expects specific normalization constants and input sizes. (3) LR too high: one hot epoch destroys pretrained features ("catastrophic forgetting") — warm up the new head first with the backbone frozen, then unfreeze. Bonus: weight decay applied to the head only vs everywhere changes effective regularization.

Q18. Why does translation equivariance differ from invariance, and where does each come from?

Equivariance: shift input → output shifts correspondingly; convolution provides this by construction (same kernel everywhere). Invariance: shift input → output unchanged; comes from aggregation — pooling locally (Listing 2: 34% of maxpool outputs survive a shift that changes 100% of pixels), global average pooling fully. A classifier wants invariance (cat anywhere = cat); a detector wants equivariance preserved (location matters). Note the caveat: padding, stride aliasing, and downsampling break exact equivariance in real nets.

Q19. Compute parameters: conv layer, 5×5 kernels, 32 input channels, 64 output channels, plus bias. And its FLOPs on a 112×112 output?

Weights: $64 \cdot (32 \cdot 5 \cdot 5) = 51{,}200$; biases 64 → 51,264 parameters. FLOPs (multiply-adds): output positions × kernel volume × $C_{out} = 112^2 \cdot (32 \cdot 25) \cdot 64 \approx 6.4 \times 10^9$ MACs. The contrast to remember: parameters are independent of spatial size; FLOPs scale with it — why efficiency papers report both and why early high-resolution layers dominate compute while late wide layers dominate parameters.

Q20. Interviewer: "Design a CNN for 32×32 CIFAR-10 from this chapter's parts." Sketch it and justify.

Stem: 3×3 conv, 64 channels (no aggressive downsampling — the image is tiny). Body: 3 stages of residual blocks (2-3 each) at 64/128/256 channels, downsampling by stride-2 conv between stages; BN + ReLU per conv; receptive field check: ~3 stages of stride 2 → final units see the whole image. Head: global average pooling → dense 10. Training: He init, SGD+momentum or AdamW, cosine + warmup, augmentation (crops/flips), label smoothing optional; ~5-10M params. Justify each part by its chapter mechanism: skips for depth, GAP against dense bloat, small kernels per VGG argument.

Q21. What are dilated (atrous) convolutions and when do they beat pooling?

The kernel samples the input with gaps: dilation d inserts d−1 zeros between taps, so a 3×3 kernel with d=2 covers 5×5 with 9 weights. Receptive field grows exponentially with stacked increasing dilations, *without* downsampling — the win for dense prediction (segmentation, speech/WaveNet) where output resolution must match input. Beats pooling when you need context AND per-pixel resolution; costs: gridding artifacts if dilation rates share factors, and no invariance benefit.

Q22. How does a CNN's learned feature hierarchy look, layer by layer, and what's the evidence?

Layer 1: oriented edges and color blobs (visualizable directly as RGB kernels — Listing 3's 80-parameter net already shows oriented structure); middle layers: textures, motifs, parts (corners, eyes, wheels); late layers: object-level, task-specific detectors. Evidence: activation-maximization visualizations, deconv/feature-inversion studies (Zeiler-Fergus), and the transfer-learning gradient itself — early layers transfer across any visual task, late layers don't (Chapter 11's global-vs-local flavor of the same observation).

Q23. Why is global average pooling preferred over flattening into dense layers?

Flatten→dense couples the head to a specific spatial size and dominates the parameter budget (VGG: ~90% of 138M in three dense layers). GAP averages each channel to one number: zero parameters, any input size, strong regularization (no memorizing positional templates), and it preserves the channel-as-detector semantics — the head reads "how much of each feature appeared." GoogLeNet introduced it; effectively every architecture since keeps it.

Q24. Your CNN trains well but test images at slightly different scales/positions fail. Diagnose and fix.

The net learned position/scale-specific templates — insufficient built-in invariance for the deployment distribution. Fixes in order of cheapness: train-time augmentation matching the deployment variation (random crops, scales — Chapter 13); test-time augmentation (average predictions over crops/flips); architectural — more pooling/striding for position, feature-pyramid or multi-scale inputs for scale; check receptive field actually covers the object at all scales (Listing 2 arithmetic). Also verify preprocessing (resize/crop policy) matches between train and serve — the most common real cause.

Q25. Transposed convolution: what is it, where is it used, and what artifact does it cause?

The gradient of convolution used as a forward op: it upsamples — each input value scatters a kernel-shaped contribution into a larger output (learnable upsampling). Used in segmentation decoders, GAN generators, autoencoders. Artifact: checkerboard patterns when kernel size isn't divisible by stride (overlapping scatter accumulates unevenly); standard fix is resize (nearest/bilinear) + ordinary conv, or kernel/stride pairs that tile evenly.

Q26. How would you compress a trained CNN for edge deployment? Chapter-native options first.

Architecture substitution: depthwise-separable backbone (8-9x cheaper, Listing 4), bottlenecks, GAP head, compound-downscale (EfficientNet-lite direction). Then the standard compression stack: channel pruning (remove low-importance filters, fine-tune), quantization (fp32→int8: 4x memory, big speedup; usually <1% accuracy with calibration), knowledge distillation (train the small net on the big net's soft outputs), weight sharing/low-rank factorization (the 1x1 trick generalized). Measure FLOPs, latency on target hardware, and accuracy — they trade differently (Chapter 27 continues to serving).

Q27. Why can a CNN trained on ImageNet transfer to medical X-rays at all, and what limits it?

Early/mid features — edges, textures, blobs, local contrast patterns — are statistics of natural images generally, and X-rays share them; transfer initializes those instead of learning from scratch, which matters most when labeled medical data is scarce. Limits: late layers encode natural-object semantics useless for pathology; grayscale/intensity statistics differ (BN stats need re-estimation); and the bigger the domain gap, the deeper the fine-tuning required (Listing 6's lesson at miniature scale). Empirically transfer still helps X-ray tasks, but less than for near-domain targets, and self-supervised pretraining on unlabeled medical images can beat ImageNet init.

Q28. CNNs vs vision transformers — when is the convolutional inductive bias a win, when a limit?

CNN priors (locality, weight sharing, translation equivariance) are strong data-efficiency levers: on small/medium datasets CNNs train easily where ViTs overfit or need heavy augmentation/distillation. At web scale the priors become a ceiling: ViTs, with weaker assumptions and global attention from layer 1, learn the useful structure from data and surpass CNNs (Chapter 17 details ViT). Convergent evidence: ConvNeXt modernized CNNs to match ViTs by borrowing their training recipes, and ViTs at small scale borrow conv stems — the bias is a dial, not a dogma. Answer template: name the prior, connect to data regime, cite one architecture per side.